

인공지능의 이해

Understanding Artificial Intelligence

유준수 (Joon Soo Yoo, Ph.D.)

Fall 2026

Defense & Advanced Security Lab (DAS Lab)

Department of Advanced Defense Technology and Industry

Jeonbuk National University

Neural Networks

From Logistic Regression to Neural Networks

Week 6

1. From Logistic Regression to Neural Networks

지난 시간에 Logistic Regression을 살펴보았다.

Logistic Regression은 먼저

$$z = w^T x + b$$

를 계산하고,

$$\hat{y} = \sigma(z)$$

를 통해 Probability를 계산한다.

즉,

Features $\rightarrow$ Weighted Sum $\rightarrow$ Sigmoid $\rightarrow$ Probability

핵심

Neural Network에서도 Weight, Bias, Activation Function이라는 매우 비슷한 구조가 사용된다.

1-1. What Does Logistic Regression Learn?

예를 들어 두 개의 Feature가 있다고 하자.

$$x_1 = \text{Bill Length}$$

$$x_2 = \text{Flipper Length}$$

Logistic Regression은

$$z = w_1x_1 + w_2x_2 + b$$

에서 적절한

$$w_1, w_2, b$$

를 Training Data로부터 학습한다.

핵심

Training은 Prediction을 잘 할 수 있도록 적절한 Weight와 Bias를 찾는 과정이다.

2. Is Logistic Regression Nonlinear?

Logistic Regression에서는 Sigmoid Function을 사용한다.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid는 Nonlinear Function이다.

그렇다면 Logistic Regression도 Nonlinear Classification을 할 수 있을까?

질문

Nonlinear Function을 사용한다는 것과 Nonlinear Decision Boundary를 만든다는 것은 같은 의미일까?

2-1. Decision Boundary of Logistic Regression

Threshold가 0.5라고 생각해 보자.

Positive Class의 조건은

$$\sigma(w^T x + b) \geq 0.5$$

이다.

Sigmoid Function의 특성 때문에 이는

$$w^T x + b \geq 0$$

와 같다.

두 개의 Feature가 있다면 Decision Boundary는

$$w_1 x_1 + w_2 x_2 + b = 0$$

이다.

핵심

Logistic Regression은 Nonlinear Sigmoid를 사용하지만, 원래 Feature Space에서의 Decision Boundary는 Linear하다.

3. When a Straight Line Is Not Enough

어떤 Classification Problem은 하나의 직선으로 쉽게 구분할 수 있다.

하지만 Data가 복잡하게 분포한다면 어떨까?

Linear Decision Boundary



Simple Classification Problems

하지만

Complex Data



A Straight Line May Not Be Enough

핵심

복잡한 Pattern을 학습하기 위해서는 보다 유연한 Model이 필요할 수 있다.

3-1. A Simple Nonlinear Problem: XOR

대표적인 Example이 XOR Problem이다.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

두 Class를 하나의 직선으로 완벽하게 나누기 어렵다.

질문

하나의 Linear Decision Boundary로 부족하다면, 여러 개의 계산을 연결하면 어떨까?

4. Artificial Neuron

Neural Network의 기본 단위를 Artificial Neuron이라고 생각할 수 있다.

입력이

$$x_1, \quad x_2, \quad x_3$$

라면 먼저

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

를 계산한다.

그리고 Activation Function을 적용한다.

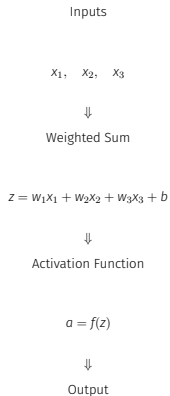
$$a = f(z)$$

핵심

Neuron은 Input을 Weight와 함께 결합하고, Activation Function을 통해 Output을 만든다.

4-1. Inside a Neuron

Neuron의 계산을 단계별로 보면 다음과 같다.



핵심

Neuron 하나의 계산 자체는 Logistic Regression에서 보았던 계산과 매우 비슷하다.

4-2. What Do Weights Mean?

각 Input에는 Weight가 연결되어 있다.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

예를 들어,

Input	Weight
x_1	w_1
x_2	w_2
x_3	w_3

Weight가 달라지면 각 Input이 Output에 미치는 영향도 달라진다.

핵심

Neural Network Training의 중요한 목적 중 하나는 적절한 Weight와 Bias를 학습하는 것이다.

5. Activation Function

Weighted Sum을 계산한 뒤 Activation Function을 적용한다.

$$z = w^T x + b$$

$$a = f(z)$$

대표적인 Activation Function에는

- ▶ Sigmoid
- ▶ Tanh
- ▶ ReLU

가 있다.

핵심

Activation Function은 Neuron의 계산에 Nonlinearity를 추가할 수 있다.

5-1. ReLU Activation Function

Deep Learning에서 널리 사용되는 Activation Function 중 하나가 ReLU이다.

$$\text{ReLU}(z) = \max(0, z)$$

예를 들어,

$$\text{ReLU}(-3) = 0$$

$$\text{ReLU}(2) = 2$$

$$\text{ReLU}(5) = 5$$

이다.

즉,

Negative Input $\rightarrow 0$

Positive Input $\rightarrow$ 그대로 전달

핵심

ReLU는 간단하지만 Neural Network에서 매우 중요한 Activation Function이다.

5-2. Why Do We Need Activation Functions?

Activation Function이 없다면 어떻게 될까?

두 개의 Linear Layer를 생각해보자.

$$h = W_1x$$

$$y = W_2h$$

그러면

$$y = W_2W_1x$$

이다.

즉,

$$W_2W_1 = W'$$

라고 하면

$$y = W'x$$

가 된다.

중요

Linear Transformation을 여러 번 연결해도 결국 하나의 Linear Transformation으로 표현할 수 있다.

5-3. Nonlinearity Matters

Activation Function을 사용하면

$$h = f(W_1x + b_1)$$

그리고

$$y = g(W_2h + b_2)$$

와 같은 구조를 만들 수 있다.

즉,

Linear Transformation



Nonlinear Activation



Linear Transformation



Nonlinear Activation

핵심

Nonlinear Activation을 사용하면 여러 Layer를 연결하여 복잡한 Pattern을 표현할 수 있다.

6. From Neurons to Layers

Neuron을 여러 개 모으면 하나의 Layer를 만들 수 있다.

예를 들어 Input이

$$x_1, \quad x_2, \quad x_3$$

이고 Hidden Neuron이 네 개라면,

$$(x_1, x_2, x_3) \rightarrow (h_1, h_2, h_3, h_4)$$

와 같이 표현할 수 있다.

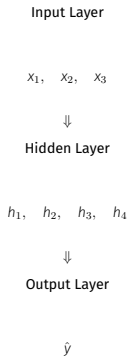
각 Hidden Neuron은 서로 다른 Weight와 Bias를 학습할 수 있다.

핵심

여러 Neuron이 서로 다른 방식으로 Input Information을 변환할 수 있다.

6-1. A Simple Neural Network

가장 기본적인 Neural Network는 다음과 같이 생각할 수 있다.



핵심

Neural Network는 여러 Neuron과 Layer를 연결하여 Input으로부터 Output을 계산한다.

6-2. Why Is It Called a Hidden Layer?

Neural Network에는 일반적으로

- ▶ Input Layer
- ▶ Hidden Layer
- ▶ Output Layer

가 있다.

Input은 우리가 제공하는 Data이고, Output은 Model이 만들어내는 Prediction이다.

하지만 Hidden Layer의 값은 Training Data에서 직접 주어지는 정답이 아니다.

핵심

Hidden Layer는 Input과 Output 사이에서 Model이 내부적으로 학습하는 Representation을 만든다.

7. Neural Network for Penguin Classification

이전에 사용했던 Penguin Classification을 생각해보자.

Input Feature가

- ▶ Bill Length
- ▶ Flipper Length
- ▶ Body Mass

라고 하자.

그러면 Neural Network를

3 Input Features



Hidden Neurons



Output



Penguin Class

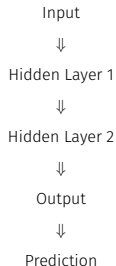
와 같이 구성할 수 있다.

핵심

Neural Network도 결국 Input Data를 받아 Prediction을 만드는 Machine Learning Model이다.

8. Forward Propagation

Input에서 시작하여 각 Layer의 계산을 순서대로 수행해보자.



이처럼 Input에서 Output 방향으로 계산을 진행하는 것을 **Forward Propagation**이라고 한다.

핵심

Forward Propagation은 현재 Weight와 Bias를 이용하여 Input으로부터 Model의 Prediction을 계산하는 과정이다.

8-1. Computing One Hidden Neuron

두 개의 Input Feature가 있다고 생각해보자.

$$x_1, \quad x_2$$

Hidden Layer의 첫 번째 Neuron은 자신의 Weight와 Bias를 이용하여

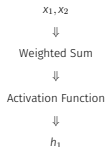
$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

를 계산한다.

그리고 Activation Function을 적용한다.

$$h_1 = f(z_1)$$

즉,



핵심

Hidden Neuron도 Input의 Weighted Sum을 계산하고 Activation Function을 적용하여 자신의 Output을 만든다.

8-2. Computing a Hidden Layer

Hidden Layer에 두 개의 Neuron이 있다고 하자.

첫 번째 Neuron은

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

$$h_1 = f(z_1)$$

를 계산한다.

두 번째 Neuron은 서로 다른 Weight와 Bias를 사용하여

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

$$h_2 = f(z_2)$$

를 계산한다.

따라서 Hidden Layer의 Output은

$$(h_1, h_2)$$

가 된다.

핵심

같은 Input을 받더라도 각 Neuron은 서로 다른 Weight와 Bias를 사용하여 서로 다른 Output을 만들 수 있다.

8-3. From One Layer to the Next

첫 번째 Hidden Layer가

$$h_1, \quad h_2$$

를 만들었다고 하자.

다음 Layer에서는 이 값들이 새로운 Input이 된다.

예를 들어 다음 Neuron은

$$z_3 = w_{31}h_1 + w_{32}h_2 + b_3$$

를 계산하고,

$$h_3 = f(z_3)$$

를 출력한다.

즉,

Original Input $(x_1, x_2) \rightarrow$ Hidden Layer 1 $(h_1, h_2) \rightarrow$ Hidden Layer 2 $(h_3, \dots)$

핵심

한 Layer의 Output이 다음 Layer의 Input이 된다.

8-4. From Hidden Layers to Prediction

마지막 Hidden Layer의 Output도 Output Layer의 Input으로 사용된다.

Binary Classification이라고 생각하면 Output Neuron은 예를 들어

$$z_{\text{out}} = v_1 h_1 + v_2 h_2 + b_{\text{out}}$$

를 계산하고,

$$\hat{y} = \sigma(z_{\text{out}})$$

를 출력할 수 있다.

예를 들어

$$\hat{y} = 0.87$$

이라면 Positive Class에 대한 Probability로 해석할 수 있다.

핵심

Forward Propagation에서는 각 Layer의 Output이 다음 Layer로 전달되고, 마지막 Output Layer에서 Prediction을 만든다.

8-5. Prediction Is Not Enough

Forward Propagation을 통해 최종 Prediction을 만들었다.

예를 들어 실제 정답이

$$y = 1$$

인데 Model의 Prediction이

$$\hat{y} = 0.2$$

라면 좋은 Prediction일까?

반대로

$$\hat{y} = 0.95$$

라면 어떨까?

Prediction을 만드는 것만으로는 Model이 얼마나 잘하고 있는지 알 수 없다.

질문

Model의 Prediction이 실제 정답과 얼마나 다른지를 어떻게 측정할 수 있을까?

9. Loss Function

Prediction과 실제 정답의 차이를 측정하기 위해 Loss Function을 사용한다.

$$Loss = L(y, \hat{y})$$

여기서

- ▶ y : Actual Value
- ▶ $\hat{y}$: Model Prediction

이다.

일반적으로 Prediction이 정답에 가까울수록 Loss는 작아진다.

핵심

Loss Function은 Model의 Prediction이 얼마나 틀렸는지를 하나의 값으로 나타낸다.

9-1. Example: Prediction and Loss

실제 정답이 Positive라고 생각해보자.

$$y = 1$$

두 Model의 Prediction이 다음과 같다고 하자.

Prediction	Interpretation
$\hat{y} = 0.95$	Actual Value에 가까움
$\hat{y} = 0.20$	Actual Value에서 멀

일반적으로

$$L(1, 0.95) < L(1, 0.20)$$

가 되기를 원한다.

핵심

좋은 Prediction은 작은 Loss를, 나쁜 Prediction은 큰 Loss를 갖도록 Loss Function을 정의한다.

9-2. What Is the Goal of Training?

Neural Network의 Prediction은 Weight와 Bias에 따라 달라진다.

Weights / Biases $\rightarrow$ Prediction $\rightarrow$ Loss

따라서 Training의 목표는

Find Weights and Biases that reduce the Loss

이다.

수학적으로는 다음과 같이 생각할 수 있다.

$$\min_{W,b} L$$

핵심

Neural Network Training은 Loss를 작게 만드는 Parameter를 찾는 과정이다.

10. How Do We Reduce the Loss?

현재 Weight에서 Loss가 크다고 생각해보자.

그러면 다음 질문이 필요하다.

“Weight를 어느 방향으로 바꾸면
Loss가 감소할까?”

이를 알아보기 위해 Loss의 변화 방향을 살펴볼 수 있다.

핵심

Training에서는 Loss가 감소하는 방향을 찾고, 그 방향으로 Weight를 조금씩 수정한다.

10-1. Gradient

Loss는 Weight에 따라 달라진다.

$$L = L(w)$$

Weight를 조금 변화시켰을 때 Loss가 어떻게 변하는지를 생각할 수 있다.

$$\frac{\partial L}{\partial w}$$

이 값은 현재 Weight 주변에서 Loss가 어느 방향으로 변화하는지 알려준다.

핵심

Gradient는 Parameter를 변화시켰을 때 Loss가 어떻게 변화하는지 알려주는 Information이다.

10-2. Gradient Descent

Gradient를 이용하여 Loss가 감소하는 방향으로 Weight를 수정할 수 있다.

대표적인 방법이 Gradient Descent이다.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

여기서

- ▶ w : 현재 Weight
- ▶ $\frac{\partial L}{\partial w}$: Gradient
- ▶ η : Learning Rate

이다.

핵심

Gradient Descent는 Gradient를 이용하여 Loss가 감소하는 방향으로 Parameter를 업데이트한다.

10-3. Intuition of Gradient Descent

Gradient Descent는 산을 내려가는 과정으로 생각할 수 있다.

Current Position



Check the Slope



Move Downhill



Lower Position

Neural Network에서는

Current Weights



Compute Gradient



Update Weights



Lower Loss

핵심

Training은 한 번에 정답을 찾는 것이 아니라, Loss를 줄이는 방향으로 Parameter를 반복해서 수정한다.

10-4. Learning Rate

Gradient Descent의 Update 식을 다시 보자.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Learning Rate η 는 한 번의 Update에서 얼마나 크게 이동할지를 결정한다.

Learning Rate	Possible Behavior
Too Small	Training이 매우 느릴 수 있음
Appropriate	Loss가 안정적으로 감소할 수 있음
Too Large	좋은 값을 지나치거나 불안정할 수 있음

핵심

Learning Rate는 Model이 Training으로 직접 학습하는 Parameter가 아니라 사람이 설정하는 Hyperparameter이다.

11. Many Weights in a Neural Network

실제 Neural Network에는 하나가 아니라 많은 Weight가 존재한다.

예를 들어,

$$w_1, w_2, w_3, \dots, w_n$$

이 있다면 각각에 대해

$$\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n}$$

을 알아야 한다.

즉 각 Weight가 Loss에 어떤 영향을 주었는지를 계산해야 한다.

질문

여러 Layer에 있는 수많은 Weight의 Gradient를 어떻게 효율적으로 계산할 수 있을까?

12. Backpropagation

Neural Network는 Forward Propagation으로 Prediction을 계산한다.

Input → Hidden Layers → Prediction

Prediction과 Actual Value를 비교하여 Loss를 계산한다.

그 다음 Output 쪽에서 Input 방향으로 각 Parameter의 Gradient를 계산한다.

Loss → Output Layer → Hidden Layers

핵심

Backpropagation은 Loss로부터 각 Weight와 Bias의 Gradient를 계산하는 방법이다.

12-1. Forward Propagation vs. Backpropagation

두 과정을 비교해보자.

Process	Main Idea
Forward Propagation	현재 Parameter를 이용하여 Prediction과 Loss를 계산한다.
Backpropagation	Loss를 이용하여 각 Parameter의 Gradient를 계산한다.

따라서,

Forward: Input $\rightarrow$ Prediction $\rightarrow$ Loss

Backward: Loss $\rightarrow$ Gradients

핵심

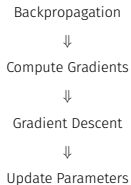
Forward Propagation은 Prediction을 계산하고, Backpropagation은 Parameter를 어떻게 수정해야 할지 필요한 Gradient를 계산한다.

12-2. Backpropagation vs. Gradient Descent

Backpropagation과 Gradient Descent는 같은 과정이 아니다.

Method	Role
Backpropagation	각 Weight와 Bias의 Gradient를 계산
Gradient Descent	계산된 Gradient를 이용하여 Parameter를 Update

즉,

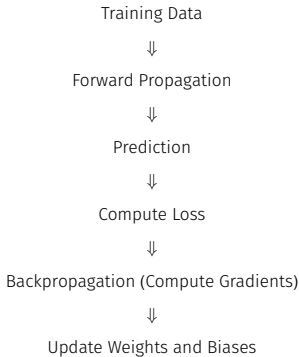


핵심

Backpropagation은 Gradient를 계산하고, Gradient Descent는 그 Gradient를 이용하여 Parameter를 업데이트한다.

13. The Neural Network Training Loop

지금까지의 과정을 하나로 연결해보자.



핵심

Neural Network Training은 이 과정을 반복하면서 Loss를 줄여나간다.

13-1. One Training Step

한 번의 Training Step을 생각해보자.

1. Training Data를 Network에 입력한다.
2. Forward Propagation으로 Prediction을 계산한다.
3. Actual Value와 비교하여 Loss를 계산한다.
4. Backpropagation으로 Gradient를 계산한다.
5. Gradient를 이용하여 Weight와 Bias를 Update한다.

그리고 이 과정을 반복한다.

핵심

Prediction → Error 확인 → Parameter 수정의 과정이 반복되면서 Model이 학습된다.

13-2. What Is an Epoch?

Training에서는 같은 Dataset을 이용하여 Parameter Update를 여러 번 수행한다.

전체 Training Data를 한 번 모두 사용하여 학습하는 것을

1 Epoch

이라고 한다.

예를 들어,

Epoch 1
Epoch 2
Epoch 3
⋮
Epoch 100

과 같이 Training을 반복할 수 있다.

핵심

Epoch는 전체 Training Dataset을 한 번 학습에 사용한 단위를 의미한다.

13-3. Loss During Training

Training이 잘 진행된다면 일반적으로 Loss가 감소하는 모습을 관찰할 수 있다.

예를 들어,

Epoch	Training Loss
1	0.68
10	0.52
50	0.31
100	0.22

이는 Model의 Parameter가 Training Data에 더 잘 맞도록 변화하고 있음을 의미한다.

참고

Training에서는 Epoch에 따른 Loss와 Accuracy의 변화를 자주 관찰한다.

14. Parameters and Hyperparameters

이전에 배운 Parameter와 Hyperparameter의 구분은 Neural Network에서도 그대로 적용된다.

Type	Examples
Parameters	Weights, Biases
Hyperparameters	Learning Rate, Number of Hidden Neurons, Epochs

Parameters는 Training 과정에서 학습된다.

Hyperparameters는 Model이나 Training 방법을 설정하기 위해 사람이 결정한다.

핵심

Neural Network에서는 많은 Weight와 Bias를 학습하고, 여러 Hyperparameter가 Training에 영향을 줄 수 있다.

15. Neural Networks and Nonlinear Boundaries

Logistic Regression의 Decision Boundary는 기본적으로 Linear하다.

하지만 Neural Network에서는

Linear Transformation



Nonlinear Activation



Linear Transformation



Nonlinear Activation

을 연결할 수 있다.

이를 통해 복잡한 Pattern과 Nonlinear Decision Boundary를 표현할 수 있다.

핵심

Neural Network의 중요한 장점은 여러 Layer와 Nonlinear Activation을 이용하여 복잡한 관계를 학습할 수 있다는 것이다.

15-1. Logistic Regression vs. Neural Network

두 Model을 비교해보자.

	Logistic Regression	Neural Network
Weight / Bias	Yes	Yes
Activation	Sigmoid	ReLU, Sigmoid, etc.
Hidden Layer	No	Yes
Decision Boundary	Linear	Can be Nonlinear
Training	Learn Parameters	Learn Parameters

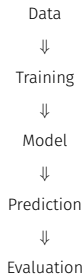
두 Model은 완전히 다른 개념이라기보다, Weight와 Bias를 학습한다는 공통점을 가진다.

핵심

Neural Network는 여러 Neuron과 Nonlinear Activation을 연결하여 더 복잡한 관계를 표현할 수 있다.

15-2. The Machine Learning Workflow Is Still the Same

Model이 Neural Network로 바뀌어도 Machine Learning의 기본 Workflow는 그대로이다.



달라진 것은 Model 내부에서 Prediction을 만드는 방법이다.

핵심

Neural Network도 Data로부터 Parameter를 학습하고, 새로운 Data에 대해 Prediction을 만드는 Machine Learning Model이다.

15-3. How Do We Evaluate a Neural Network?

Neural Network를 사용한다고 해서 새로운 Evaluation 방법만 사용해야 하는 것은 아니다.

Classification Problem이라면 이전에 배운 Metric을 그대로 사용할 수 있다.

- ▶ Confusion Matrix
- ▶ Accuracy
- ▶ Precision
- ▶ Recall
- ▶ F1 Score
- ▶ ROC / AUC

핵심

Evaluation Metric은 Model의 종류보다 해결하려는 Problem과 평가 목적에 따라 선택한다.

16. From Features to Representations

지금까지 Penguin Classification에서는 사람이 사용할 Feature를 선택했다.

예를 들어,

- ▶ Bill Length
- ▶ Flipper Length
- ▶ Body Mass

그리고 Model은 이 Feature를 이용하여 Prediction을 수행했다.

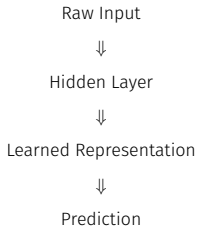
하지만 Image, Audio, Text와 같은 복잡한 Data에서는 어떤 Feature를 사용해야 할까?

질문

복잡한 Data에서도 사람이 항상 좋은 Feature를 직접 정의할 수 있을까?

16-1. Learning Representations

Neural Network의 Hidden Layer는 Input을 새로운 형태로 변환한다.



즉 Neural Network는 Prediction에 유용한 Representation을 Training 과정에서 학습할 수 있다.

핵심

Neural Network는 Output만 학습하는 것이 아니라, Prediction에 유용한 내부 Representation도 함께 학습할 수 있다.

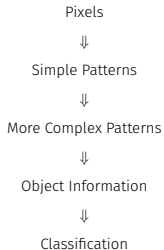
16-2. Example: Image Classification

Image Classification을 생각해보자.

Computer가 처음 받는 것은 Image를 구성하는 Pixel 값이다.

Neural Network의 여러 Layer는 이러한 Input으로부터 유용한 Pattern을 학습할 수 있다.

개념적으로 다음과 같이 생각할 수 있다.



핵심

여러 Layer를 통해 Input Data를 점점 더 유용한 Representation으로 변환할 수 있다.

17. From Neural Networks to Deep Learning

지금까지 하나의 Hidden Layer를 가진 간단한 Neural Network를 생각했다.

Input Layer



Hidden Layer



Output Layer

Hidden Layer를 여러 개 사용하면

Input → Hidden 1 → Hidden 2 → Hidden 3 → Output

와 같은 Network를 만들 수 있다.

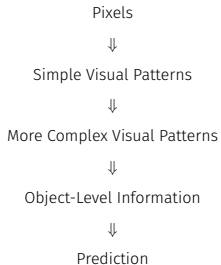
핵심

여러 Layer를 사용하는 깊은 Neural Network가 Deep Learning의 기본적인 출발점이다.

17-1. Why Go Deeper?

여러 Hidden Layer를 사용하면 각 Layer가 이전 Layer의 Representation을 다시 변환할 수 있다.

예를 들어 Image를 매우 단순화하여 생각하면,

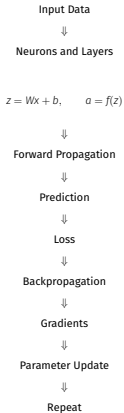


핵심

깊은 Network는 여러 단계의 Transformation을 통해 복잡한 Representation을 학습할 수 있다.

18. Putting Everything Together

Neural Network의 전체 과정을 연결해보자.



핵심

Neural Network는 Forward Propagation으로 Prediction하고, Loss와 Backpropagation을 이용하여 Parameter를 학습한다.

18-1. What Is Learned and What Is Chosen?

Neural Network의 요소를 다시 구분해보자.

Learned from Data	Chosen / Configured
Weights	Number of Layers
Biases	Number of Neurons
	Activation Function
	Learning Rate
	Number of Epochs

왼쪽은 Training 과정에서 학습되는 **Parameters**이다.

오른쪽은 Model과 Training을 설정하는 **Hyperparameters**이다.

핵심

Neural Network Training에서는 Parameter를 학습하지만, Network의 구조와 Training 방법에는 여러 Hyperparameter가 존재한다.

19. What We Learned

이번 Week에서는 Neural Network의 기본 구조와 Training 원리를 살펴보았다.

- ▶ Neural Network도 Weight와 Bias를 학습한다.
- ▶ Neuron은 Weighted Sum과 Activation Function을 이용한다.
- ▶ Nonlinear Activation은 복잡한 관계를 표현하는 데 중요하다.
- ▶ 여러 Neuron을 모아 Layer를 구성할 수 있다.
- ▶ Forward Propagation으로 Prediction을 계산한다.
- ▶ Loss Function으로 Prediction Error를 측정한다.
- ▶ Gradient는 Loss가 변화하는 방향에 대한 Information을 제공한다.
- ▶ Backpropagation으로 각 Parameter의 Gradient를 계산한다.
- ▶ Gradient Descent를 이용하여 Parameter를 Update한다.
- ▶ 이 과정을 반복하면서 Neural Network가 학습된다.

Thank You

Questions?